



# 算法分析与设计

Analysis and Design of Algorithm

## Lesson 03



# 要点回顾

## ■ 算法复杂度的概念

### ■ 时间复杂度

- 最坏情况时间复杂度  $W(n)$
- 平均情况时间复杂度  $A(n)$

### ■ 衡量标准：计量函数

## ■ 复杂度的渐近性态

- 略去低阶项所留下的主项
- 五个渐近分析记号

## ■ NP完全性理论

- 易解、难解问题

1. 渐近上界记号  $O$
2. 渐近下界记号  $\Omega$
3. 紧渐近界记号  $\Theta$
4. 非紧上界记号  $o$
5. 非紧下界记号  $\omega$

# 易解问题与难解问题

- 通常将存在**多项式时间**算法的问题看作是**易解问题** (Easy Problem) ;
  - 排序问题、查找问题、欧拉回路 → **问题规模**
- 而将需要**指数时间**算法解决的问题看作是**难解问题** (Hard Problem) 。
  - TSP问题、Hanio问题、Hamilton回路问题

# 易解问题与难解问题

## ■ 为什么把多项式时间复杂性作为易解问题和难解问题的分界线?

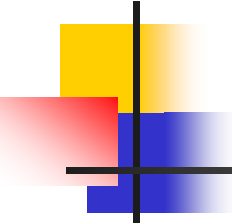
### ■ 多项式函数与指数函数的增长率有本质的差别

|                 | $n$     | $n \log_2 n$ | $n^2$   | $n^3$        | $1.5^n$      | $2^n$           | $n!$            |
|-----------------|---------|--------------|---------|--------------|--------------|-----------------|-----------------|
| $n = 10$        | < 1 sec | < 1 sec      | < 1 sec | < 1 sec      | < 1 sec      | < 1 sec         | 4 sec           |
| $n = 30$        | < 1 sec | < 1 sec      | < 1 sec | < 1 sec      | < 1 sec      | 18 min          | $10^{25}$ years |
| $n = 50$        | < 1 sec | < 1 sec      | < 1 sec | < 1 sec      | 11 min       | 36 years        | very long       |
| $n = 100$       | < 1 sec | < 1 sec      | < 1 sec | 1 sec        | 12,892 years | $10^{17}$ years | very long       |
| $n = 1,000$     | < 1 sec | < 1 sec      | 1 sec   | 18 min       | very long    | very long       | very long       |
| $n = 10,000$    | < 1 sec | < 1 sec      | 2 min   | 12 days      | very long    | very long       | very long       |
| $n = 100,000$   | < 1 sec | 2 sec        | 3 hours | 32 years     | very long    | very long       | very long       |
| $n = 1,000,000$ | 1 sec   | 20 sec       | 12 days | 31,710 years | very long    | very long       | very long       |

# 不可解问题

- 不能用计算机求解(不论耗费多少时间)的问题称为**不可解问题**(Unsolvable Problem)
  - **图灵**停机问题(Turing Halting Problem)

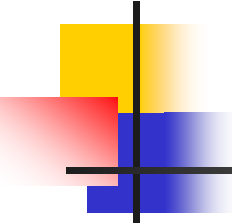




# P类问题和NP类问题

---

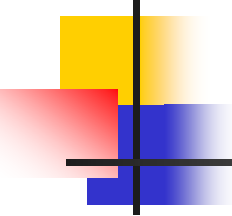
- 判定问题
- 确定性算法与P类问题
- 非确定性算法与NP类问题



# 判定问题

---

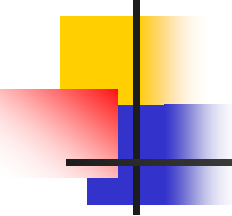
- 一个**判定问题** (Decision Problem) 是仅仅要求回答 “yes” 或 “no” 的问题
- 判定问题的重要特性——**证明比求解易**
  - 大整数因子
  - 复杂线性方程组的解



# 确定性算法与P类问题

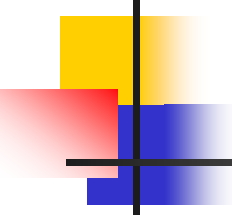
- **定义1** 设A是求解问题 $\Pi$ 的一个算法，如果在算法的整个执行过程中，每一步只有一个确定的选择，则称算法A是**确定性(Determinism)算法**
- **定义2** 如果对于某个判定问题 $\Pi$ ，存在一个非负整数 $k$ ，对于输入规模为 $n$ 的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到yes或no的答案，则该判定问题 $\Pi$ 是一个**P类(Polynomial)问题**

**所有易解问题都是P类问题**



# 非确定性算法与NP类问题

- **定义3** 设A是求解问题 $\Pi$ 的一个算法，如果算法A以如下猜测并验证的方式工作，就称算法A是**非确定性(Nondeterminism)算法**
  - **猜测阶段**：在这个阶段，对问题的输入实例产生一个任意字符串y，在算法的每一次运行时，串y的值可能不同，因此，猜测以一种非确定的形式工作；
  - **验证阶段**：在这个阶段，用一个确定性算法验证：
    - 检查在猜测阶段产生的串y是否是合适的形式，如果不是，则算法停下来并得到no；
    - 如果串y是合适的形式，则验证它是否是问题的解，如果是，则算法停下来并得到yes，否则算法停下来并得到no。

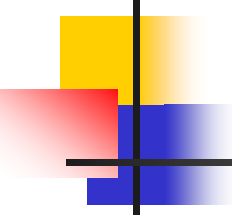


# 非确定性算法与NP类问题

- **定义4** 如果对于某个判定问题 $\Pi$ ，存在一个非负整数 $k$ ，对于输入规模为 $n$ 的实例，能够以 $O(n^k)$ 的时间运行一个非确定性算法，得到yes或no的答案，则该问题是一个**NP类 (Nondeterministic Polynomial)问题**

**关键：**存在一个**确定性算法**，能够以**多项式时间**来检查和**验证**猜测阶段所产生的答案。

**例如：**NP类问题——汉密尔顿回路问题



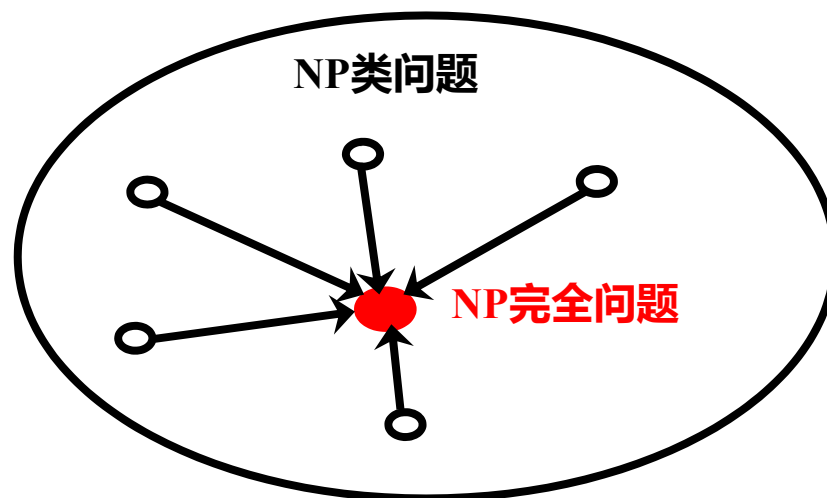
# P类和NP类问题的主要差别

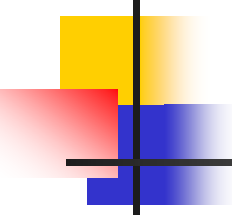
- P类问题可以用多项式时间的**确定性算法**来进行判定或求解；
- NP类问题可用多项式时间的**非确定性算法**来进行判定或求解。

$$P \subseteq NP$$

# NP完全问题

- **定义5** 令 $\Pi$ 是一个判定问题，如果问题 $\Pi$ 属于NP类问题，并且对NP类问题中的每一个问题 $\Pi'$ ，都有 $\Pi' \leq_p \Pi$ ，则称问题 $\Pi$ 是一个**NP完全问题(NP Complete Problem)**，有时把NP完全问题记为**NPC**



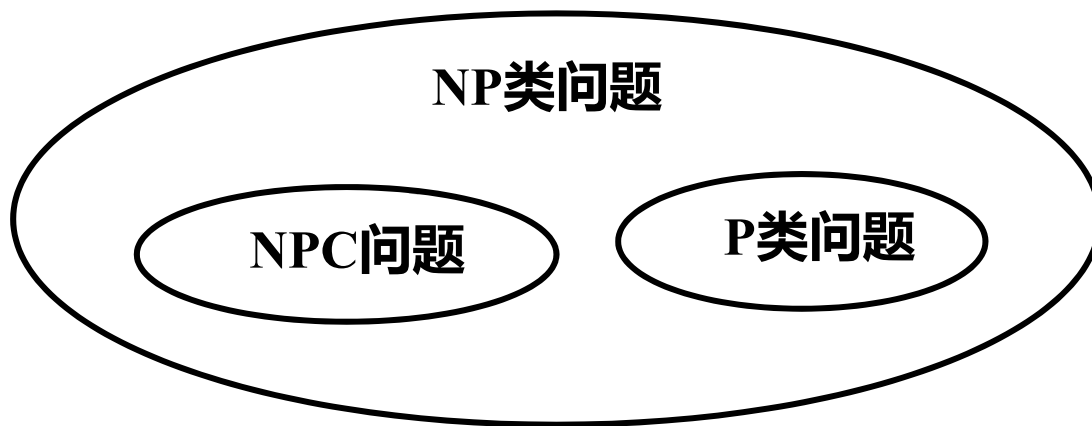


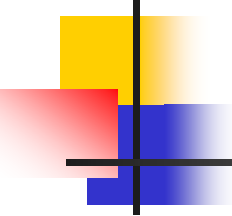
# 一些基本的NP完全问题

- **SAT问题(Boolean Satisfiability Problem)**
- **最大团问题(Maximum Clique Problem)**
- **图着色问题(Graph Coloring Problem)**
- **哈密顿回路问题(Hamiltonian Cycle Problem)**
- **TSP问题(Traveling Salesman Problem)**
- **顶点覆盖问题(Vertex Cover Problem)**
- **最长路径问题(Longest Path Problem)**
- **子集和问题(Sum of Subset Problem)**

# 三者关系

- 目前人们猜测 $P \neq NP$ ，则P类问题、NP类问题、NP完全问题之间的关系如下：





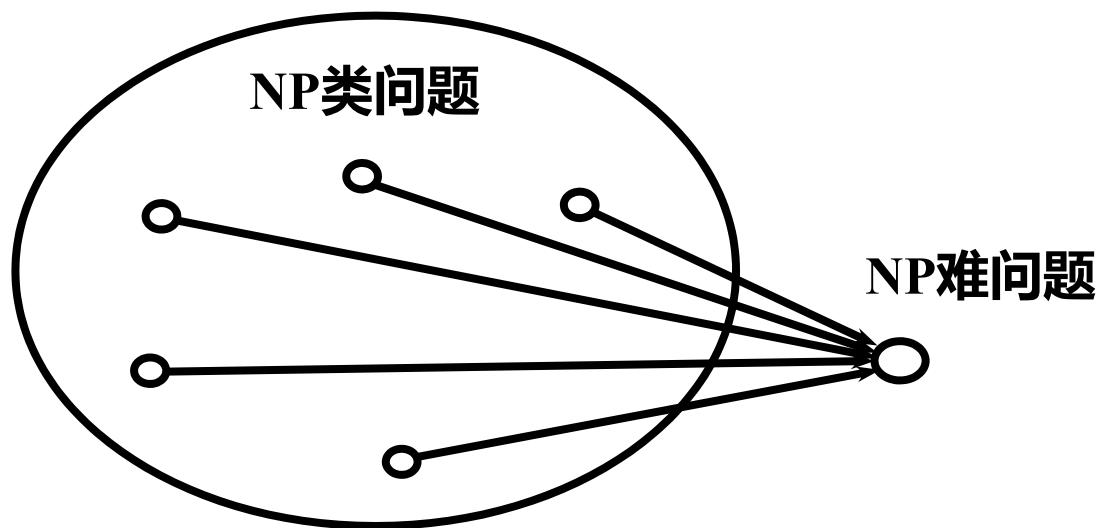
# 千禧难题

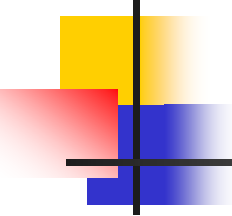
---

1. **P类问题 $\neq$ NP类问题猜想**
2. **霍奇(Hodge)猜想**
3. **庞加莱(Poincare)猜想**
4. **黎曼(Riemann)假设**
5. **杨-米尔斯(Yang-Mills)存在性和质量缺口**
6. **纳维叶-斯托克斯(Navier-Stokes)方程的存在性与光滑性**
7. **贝赫(Birch)和斯维纳通-戴尔(Swinnerton-Dyer)猜想**

# NP难问题

- **定义6** 令 $\Pi$ 是一个判定问题，如果对于NP类问题中的每一个问题 $\Pi'$ ，都有 $\Pi' \leq_p \Pi$ ，但是并不能证明问题 $\Pi$ 是NP类问题，则称判定问题 $\Pi$ 是一个**NP难问题**。



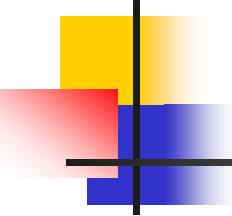


# NPC和NP难问题的区别

- 如果， $\Pi$ 是NPC问题， $\Pi'$ 是NP难问题，那么，他们之间的差别在于 $\Pi$ 必定是NP类问题，而 $\Pi'$ 不一定在NP类问题中。

**一般而言，若判定问题属于NP完全问题，则相应的最优化问题属于NP难问题。**

例如，判定图 $G = (V, E)$ 中是否存在汉密尔顿回路是NP完全问题，而求汉密尔顿回路中最短路径的TSP问题则是NP难问题。



# 第一章总结

---

- 理解算法的概念
- 理解算法、数据结构和程序的区别和联系
- 掌握描述算法的方法
- 掌握算法的计算复杂性概念
- 掌握算法渐近复杂性的数学表述
- 了解NP类问题的基本概念 (P、NP、NPC、NP-Hard)



# 第一章作业

---

## ■ 上传群里

- Tex版本: 2024-hw01.tex
- Markdown版本: 2024-hw01.md

## ■ 提交要求:

- 一份答案电子版PDF
  - 命名: 学号-姓名-第01次作业.pdf
- 电子邮件地址: (助教-陈铎文)

## ■ 截止时间:

- 一周内完成 (如: 周一课堂布置, 周日23:59前提交; 周五课堂布置, 下周四23:59前提交)



# LaTeX教程

- **TEX是斯坦福大学的教授Donald E. Knuth开发的一个功能强大的幕后排版系统。当时在撰写名为《The Art of Computer Programming》的书，由于出版商把书中的数学式子排版得很难看，决定推迟出版，自行研发一套排版系统进行排版。这个系统就是TEX系统。**
- **LaTeX：**
  - **TEX是很低阶的排版语言，对于绝大多数人来说，学起来会很吃力，而且排版工作也会变得相当繁复，难以被更多人使用，效率也不是很高。所以，一些经常用到的功能，如果我们事先定义好，到要用的时候只引用一小段代码就可以实现一个相对复杂的功能，那不仅提高了排版效率，而且版面也会清晰很多。这种事先定义好的功能，叫做宏集(macro)。**
  - **LaTeX是TEX众多宏集之一，是由Leslie Lamport编写的。**



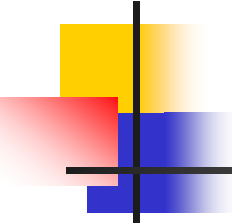
# LaTeX教程

---

## ■ 离线：CTeX

- CTeX是利用TEX排版系统中文套装的简称
- CTeX中文套装在MiKTeX的基础上增加了对中文的完整支持，集成了编辑器 WinEdt 和 PostScript 处理软件 Ghostscript 和 GSview 等主要工具
- 支持CCT和CJK两种中文TeX处理方式
- <http://www.ctex.org>

## ■ 在线：<https://www.overleaf.com/>



# 课程内容

NP完全性理论与近似算法

高级理论

随机化算法

线性规划与网络流

高级算法

递归  
分治

动态  
规划

贪心  
算法

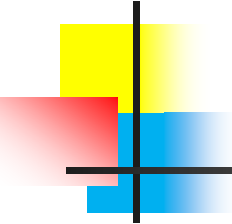
回溯与  
分支限界

基础算法

算法分析与问题的计算复杂性

基础理论

# 第二章 递归与分治策略



# 学习要点

---

- **理解**分治和递归的概念。
- **掌握**设计有效算法的分治策略。
- 通过下面的范例学习**掌握**分治策略**设计技巧**。
  - 二分搜索技术；
  - 大整数乘法；
  - Strassen矩阵乘法；
  - 棋盘覆盖；
  - 合并排序和快速排序；
  - 线性时间选择；
  - 最接近点对问题；
  - 循环赛日程表。



# 分治法的初衷

- 任何一个问题的求解时间都与其规模有关。
- 例子：

- $n$ 个元素排序：

- 当 $n=1$ ，不需计算；

- 当 $n=2$ ，只作一次即可；

- 当 $n=3$ ， ?

- ...

显然，随着 $n$ 的增加，问题也越难处理。



# 分治法

- 分治法的**设计思想**是：将一个难以直接解决的大问题，分割成一些**规模较小的相同问题**，以便各个击破，分而治之。
- 如果问题可分割成 $k$ 个子问题，且这些子问题都可解，利用这些子问题可解出原问题的解，此时，分治法是可行的。
- 由分治法产生的子问题往往是原问题的较少模式，为**递归**提供了方便。

# 递归

## ■ 定义：直接/间接调用自身的算法

阶乘函数

$$n! = \begin{cases} 1 & , n = 0 \\ n \times (n-1)! & , n > 0 \end{cases}$$

```
int Factorial(int n) {  
    if (n==0) return 1;  
    return n×Factorial(n-1)  
}
```

递归第一式给出函数的初值，非递归定义。每个递归须有非递归初始值。

第二式是用较小自变量的函数值表示较大自变量。

# 递归

## ■ 前面提到的 **Fibonacci**数列

$$F(n) = \begin{cases} 1 & , n = 0 \\ 1 & , n = 1 \\ F(n-1) + F(n-2) & , n > 1 \end{cases}$$

上述函数也可用非递归方式定义：

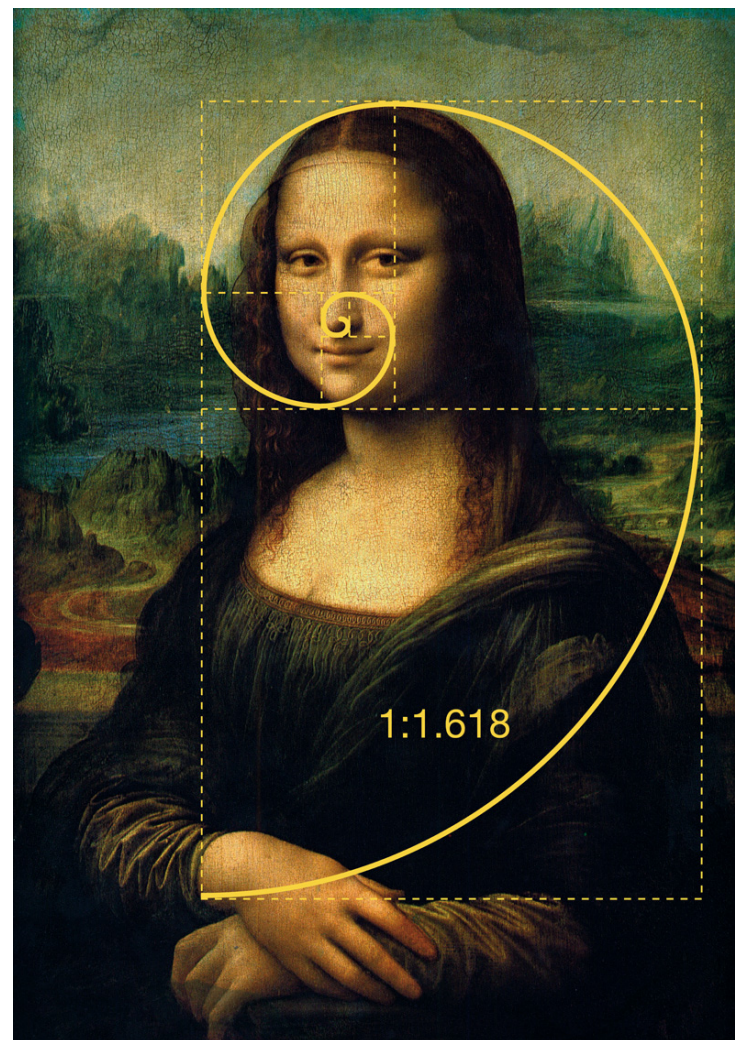
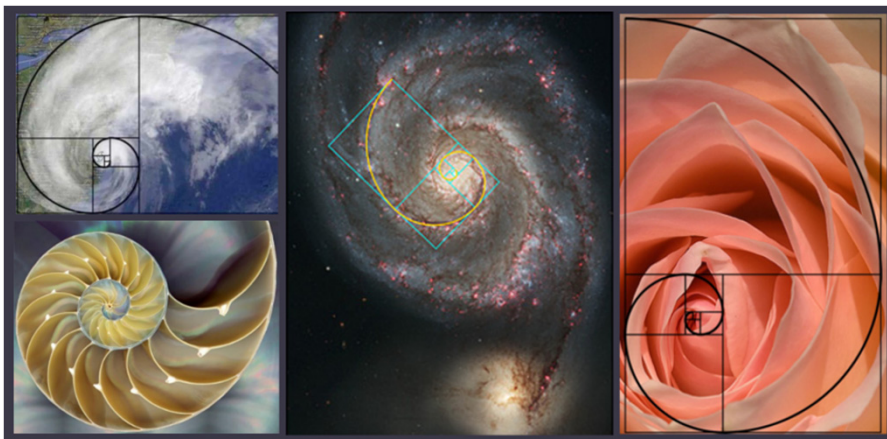
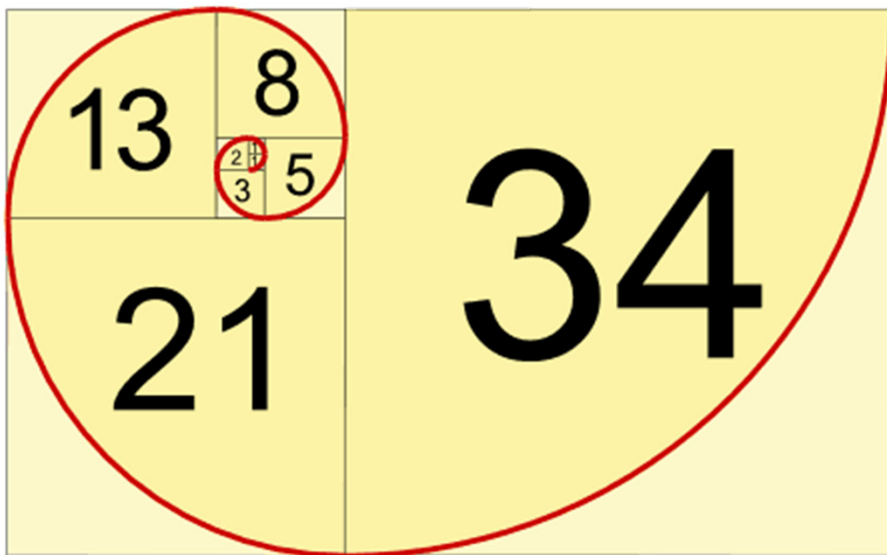
$$n! = 1 \times 2 \times 3 \times \cdots \times (n-1) \times n$$

$$F(n) = \frac{1}{\sqrt{5}} \left( \left( \frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left( \frac{1 - \sqrt{5}}{2} \right)^{n+1} \right)$$



意大利数学家  
Fibonacci  
1170-1240

# Fibonacci数列





# 双递归函数

- 一个函数与它的一个变量是由函数自身定义。

Ackerman函数

$$A(n, m): \begin{cases} A(1, 0) = 2 \\ A(0, m) = 1 & m \geq 0 \\ A(n, 0) = n + 2 & n \geq 2 \\ A(n, m) = A(A(n-1, m), m-1) & n, m \geq 1 \end{cases}$$

1.  $m=0, A(n, 0)=n+2$
2.  $m=1, A(n, 1)=$

# 双递归函数

4.  $m=3, A(n, 3) = 2^{2^{\cdot^{\cdot^2}}}$  , 其中2的层数 $n$
5.  $m=4, A(n, 4)$ 的增长速度非常快, 以至于没有适当的数学式子来表示这一函数。

■ 部分书上**减少**Ackerman函数的变量, 如:

$$A(n) \stackrel{\Delta}{=} A(n, n)$$

$$A(4) = 2^{2^{\cdot^{\cdot^2}}} \text{ (其中2的层数为65536)}$$

**这个数非常大, 无法用通常的方式来表达它!**



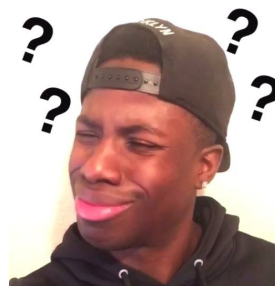
# 整数划分问题

- 将正整数 $n$ 表示成一系列正整数之和：
  - $n=n_1+n_2+\dots+n_k$  , 其中 $n_1\geq n_2\geq\dots\geq n_k\geq 1$  ,  $k\geq 1$ 。
- 正整数 $n$ 的这种表示称为正整数 $n$ 的划分。
  - **问题：**求正整数 $n$ 的不同划分**个数**。
- 例如，正整数6有如下11种不同的划分：
  - 6;
  - 5+1;
  - 4+2, 4+1+1;
  - 3+3, 3+2+1, 3+1+1+1;
  - 2+2+2, 2+2+1+1, 2+1+1+1+1;
  - 1+1+1+1+1+1。

# 整数划分问题

- 仅仅考虑一个自变量：

- 设 $p(n)$ 为正整数 $n$ 的划分数
- 难以找到递归关系

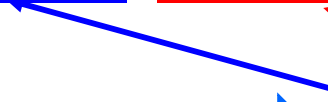


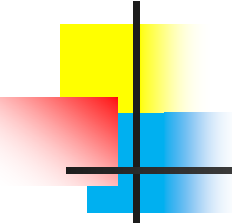
- 考虑两个自变量：

- 将最大加数 $n_1$ 不大于 $m$ 的划分个数记作 $q(n, m)$ 。

# 整数划分问题

- 可以建立 $q(n, m)$ 的如下递归关系。

- (1)  $q(n, 1) = 1, n \geq 1;$   当最大加数 $n_1$ 不大于1时, 任何正整数 $n$ 只有一种划分形式即 $n$ 个1,  $\{1, 1, 1, \dots, 1\};$
- (2)  $q(n, m) = q(n, n), m \geq n;$   最大加数 $n_1$ 实际上不能大于 $n$ 。因此,  $q(1, m) = 1。$
- (3)  $q(n, n) = 1 + q(n, n-1);$   正整数 $n$ 的划分由 $n_1 = n$ 的划分和 $n_1 \leq n-1$ 的划分组成。
- (4)  $q(n, m) = \underline{q(n-m, m)} + \underline{q(n, m-1)}, n > m > 1;$   
 正整数 $n$ 的最大加数 $n_1$ 不大于 $m$ 的划分由  
 $n_1 = m$ 的划分和 $n_1 \leq m-1$ 的划分组成

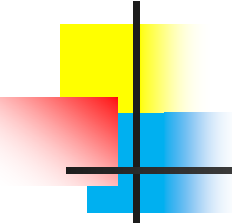


# 整数划分问题

- 可以建立 $q(n, m)$ 的如下递归关系。

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

正整数 $n$ 的划分数 $p(n) = q(n, n)$



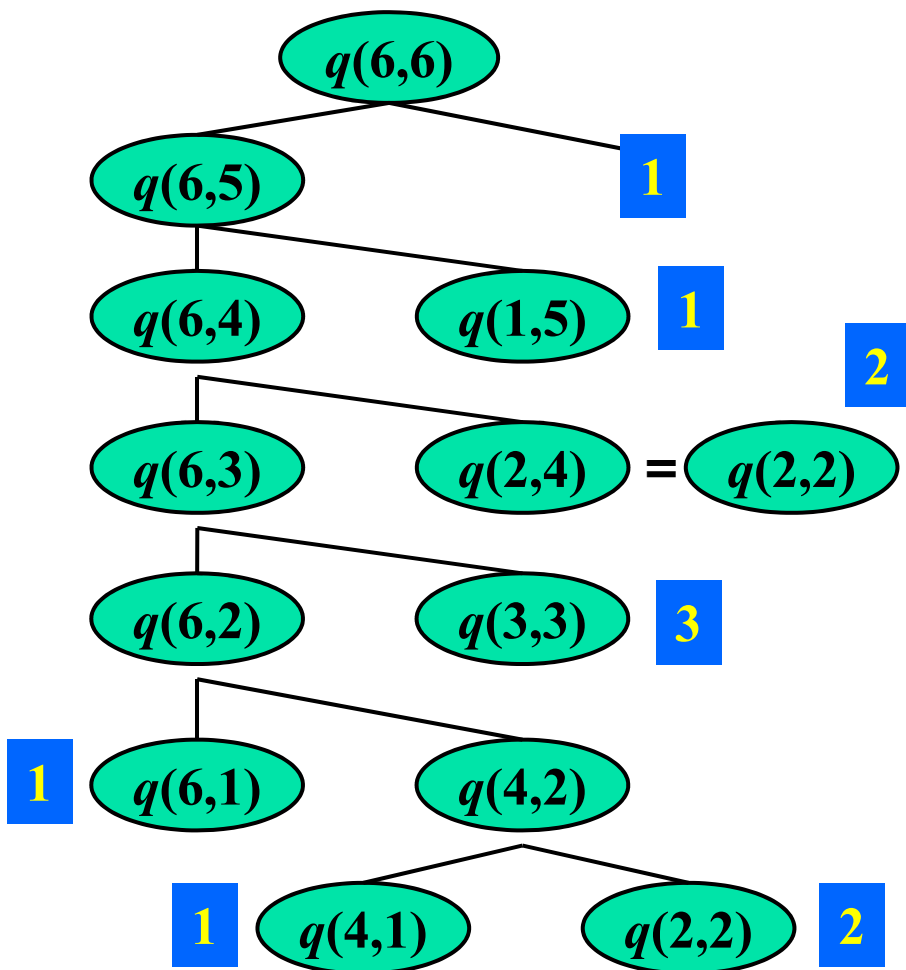
# 整数划分问题

- 可得算法伪代码为：

```
int q(int n, int m) {  
    if ((n<1) || (m<1))    return 0;  
    if ((n==1) || (m==1))  return 1;  
    if (n<m)    return q(n, n);  
    if (n==m)   return q(n, m-1)+1;  
    return q(n, m-1)+q(n-m, m);  
}
```

# 整数划分问题的递归调用

## 运行轨迹



```
int q(int n, int m) {  
    if ((n < 1) || (m < 1))  
        return 0;  
    if ((n == 1) || (m == 1))  
        return 1;  
    if (n < m)  
        return q(n, n);  
    if (n == m)  
        return q(n, m - 1) + 1;  
    return q(n, m - 1) + q(n - m, m);  
}
```

$$q(6, 6) = 1 + 1 + 2 + 3 + 1 + 1 + 2 = 11$$